Employee Attrition Prediction

Using Machine Learning

Hexovate Solutions — ML Internship Assignment

Team Manager: Ahmad Hanif — Deadline: 24 June 2026

Approach

The dataset used is the IBM HR Analytics Employee Attrition dataset containing 1,470 employee records and 35 features, sourced from Kaggle. The target variable `Attrition` is binary: Yes (employee left) or No (employee stayed). As shown in Figure 1, the class distribution is 83.88% No (1,233 employees) and 16.12% Yes (237 employees), confirming a significant class imbalance.

Figure 1: Class distribution of target variable Attrition

Preprocessing Strategy: Four constant or irrelevant columns were removed: `EmployeeCount`, `StandardHours`, `Over18`, and `EmployeeNumber`. No missing values were found across any column. Binary text columns (`Gender`, `OverTime`) were Label Encoded; multi-category columns (`BusinessTravel`, `Department`, `EducationField`, `JobRole`, `MaritalStatus`) were One-Hot Encoded, expanding the feature matrix from 31 to 49 columns. `StandardScaler` was applied to normalise all numerical features. The data was split 80/20 with stratification (Train: 1,176 rows; Test: 294 rows). SMOTE was applied exclusively to the training set after splitting, balancing it from 986 stayed / 190 left to 986 / 986. The test set remained untouched at 247 stayed / 47 left throughout all evaluations – verified by explicit row count checks confirming 1,176 + 294 = 1,470 with zero synthetic contamination.

Feature Engineering Pipeline: A second enhanced pipeline was built on top of the baseline. IQR-based outlier capping was applied to six skewed columns: `MonthlyIncome` (19,999 to 16,581), `TotalWorkingYears` (40 to 28.5), `YearsSinceLastPromotion` (15 to 7.5), `NumCompaniesWorked` (9 to 8.5), and `TrainingTimesLastYear` (6 to 4.5). Four composite features were created: `IncomePerYear` (pay fairness ratio), `SatisfactionIndex` (mean of all four satisfaction scores), `PromotionLag` (career stagnation ratio), and `YearsWithManagerRatio` (manager stability ratio), expanding the feature matrix to 53 columns.

Modelling Strategy: All four required models were trained on the SMOTE-balanced training set and evaluated on the real untouched test set. Since the cost of missing an actual leaver (false negative) is far higher than a false alarm (false positive) in an HR context, Recall and F1-Score were chosen as primary evaluation metrics rather than accuracy.

Models Used

- **Logistic Regression:** Baseline linear classifier. Chosen for interpretability; model coefficients directly map each feature's contribution to attrition probability. Performs well when feature

engineering surfaces clear linear signals.

- **Decision Tree:** Non-linear, rule-based model. Chosen for visual interpretability and ability to capture threshold-based decision rules such as: if OverTime equals Yes and JobLevel is less than 3, predict Leave.
- **Random Forest:** Ensemble of 100 decision trees. Chosen for robustness against overfitting through averaging, reducing the high variance of a single Decision Tree.
- **XGBoost:** Sequential gradient boosting classifier. Chosen for typically superior performance on tabular data by iteratively correcting errors of previous trees. Hyperparameter tuning was applied via GridSearchCV with `scale_pos_weight` to further address class imbalance.

Results Summary

Baseline Pipeline: Single Split Test Set

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.7755	0.3733	0.5957	0.4590
Decision Tree	0.7721	0.3333	0.4255	0.3738
Random Forest	0.8469	0.5500	0.2340	0.3284
XGBoost	0.8878	0.8182	0.3830	0.5217

Table 1: Baseline pipeline results on real test set (294 employees)

Baseline Pipeline: 5-Fold Stratified Cross-Validation

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.8762	0.6930	0.4218	0.5234
Decision Tree	0.7878	0.3634	0.3968	0.3780
Random Forest	0.8592	0.8727	0.1559	0.2611
XGBoost	0.8639	0.6492	0.3460	0.4492

Table 2: 5-fold stratified cross-validation averaged results (baseline pipeline)

Engineered Pipeline: Single Split Test Set

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.7857	0.3919	**0.6170**	0.4793
Decision Tree	0.7857	0.3261	0.3191	0.3226
Random Forest	0.8435	0.5217	0.2553	0.3429
XGBoost	0.8605	0.6364	0.2979	0.4058

Table 3: Engineered pipeline results. Bold indicates highest Recall at default threshold.

Tuned XGBoost: GridSearchCV

Best parameters: `n_estimators=200, max_depth=5, learning_rate=0.1, subsample=0.8, colsample_bytree`
`scale_pos_weight=5` (216 combinations, 1,080 fits). Test set: Accuracy=0.8469, Precision=0.5238, Recall=0.4681, F1=0.4944. The cross-validated F1 of 0.9389 during tuning was computed on SMOTE synthetic data and is excluded from all comparisons.

Confusion Matrices: All Models, Both Pipelines

Figure 2 shows confusion matrices for all four models across both pipelines at the default threshold of 0.50. The test set contained 47 actual leavers and 247 employees who stayed. Logistic Regression on the engineered pipeline caught the most leavers at 29 of 47, while Random Forest caught only 11 to 12 despite showing 84% accuracy – demonstrating that accuracy is a misleading metric on imbalanced data.

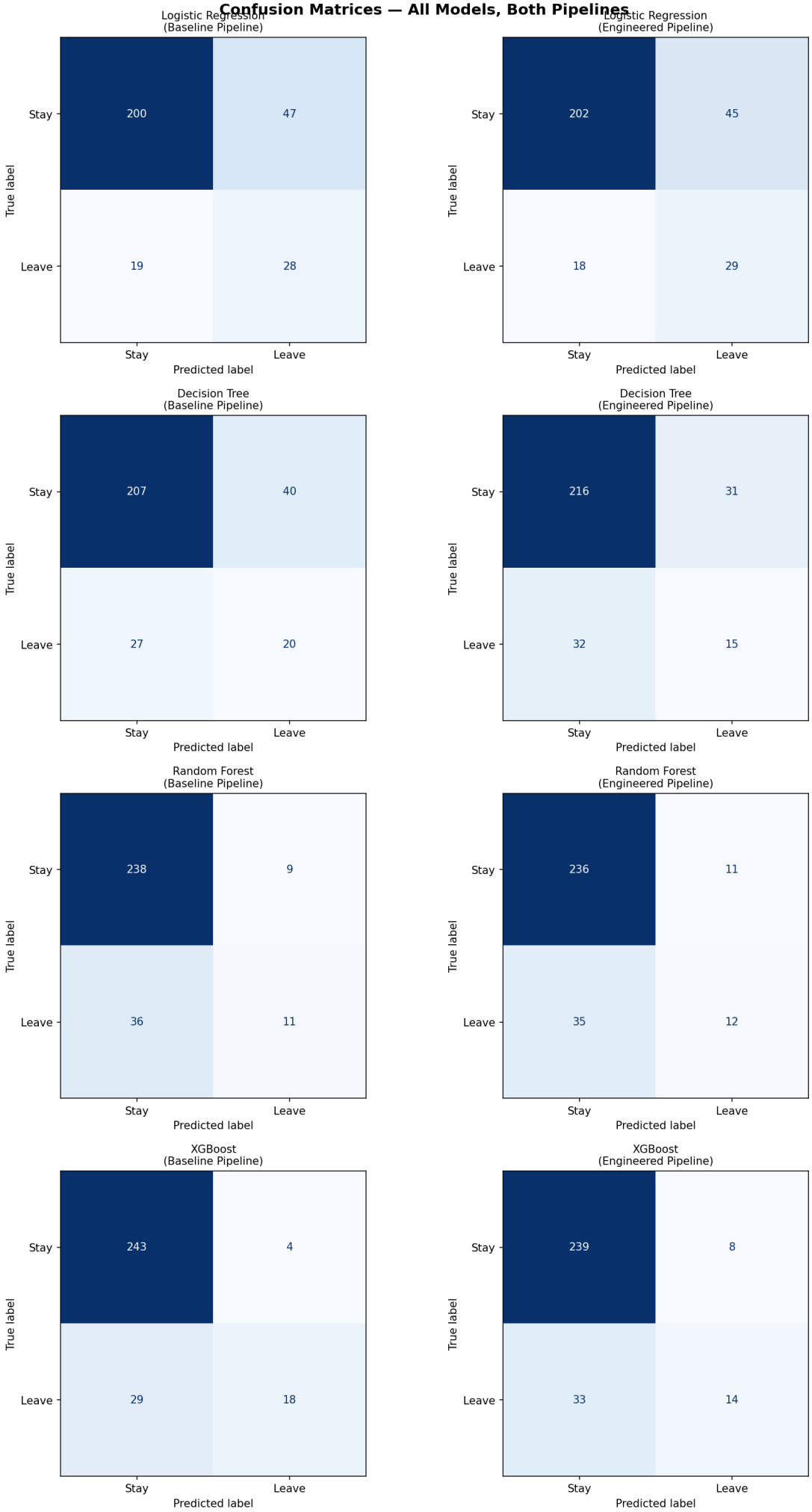

Figure 2: Confusion matrices for all models across both pipelines at default threshold 0.50

Threshold Optimisation and ROC-AUC Analysis

All models were evaluated using probability threshold tuning to find the cutoff that maximises F1-Score on the test set. Table 4 summarises the results. Figure 3 shows the confusion matrices at optimal thresholds.

Model	Optimal Threshold	Default Recall	Tuned Recall	Tuned F1
Logistic Regression	0.51	0.6170	0.6170	0.4915
Decision Tree	1.00	0.3191	0.3191	0.3226
Random Forest	0.30	0.2553	**0.7234**	**0.5231**
XGBoost	0.03	0.2979	0.7660	0.4645

Table 4: Threshold optimisation results. Bold indicates best practical tuned performance.

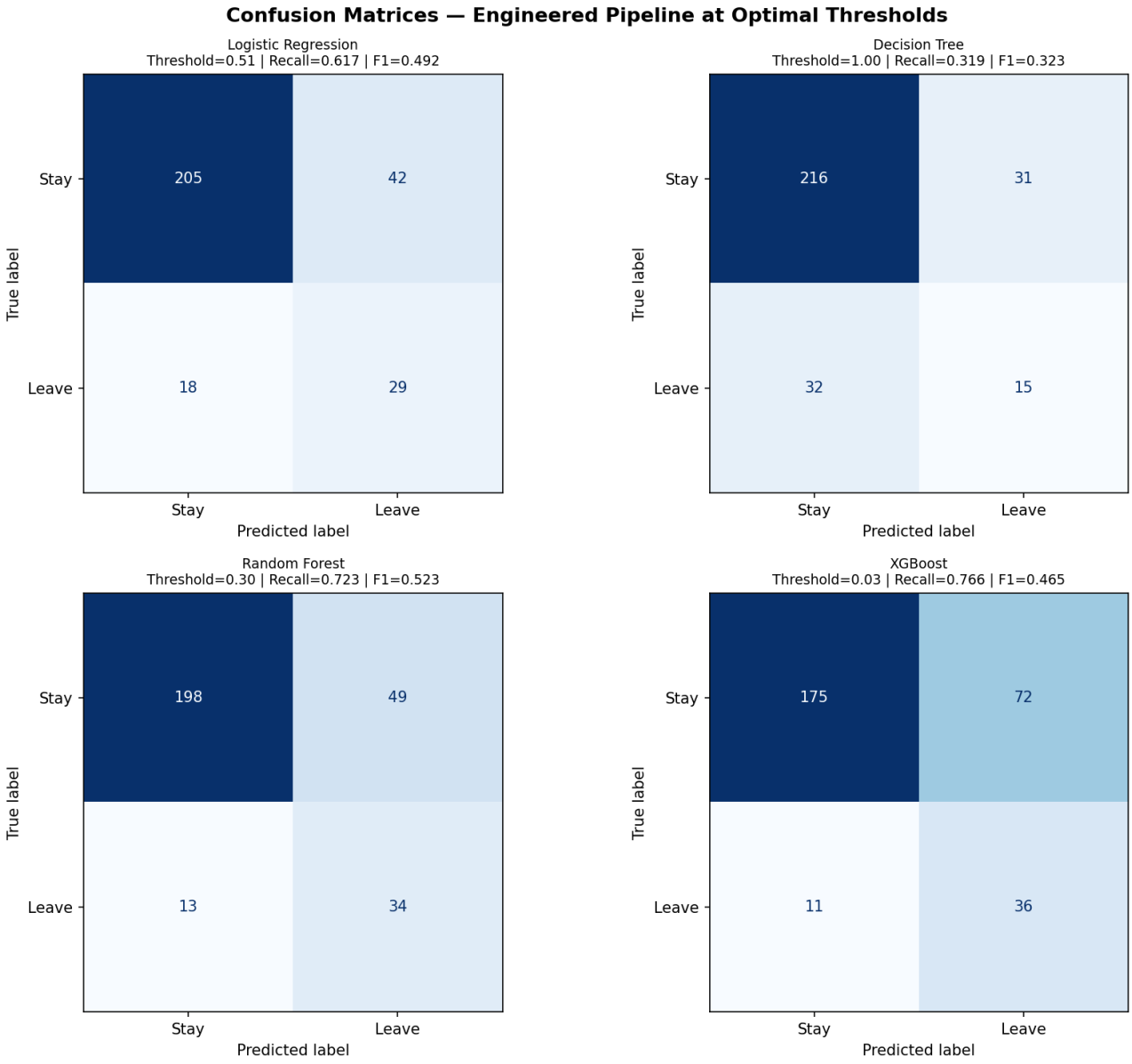

Figure 3: Confusion matrices at optimal thresholds (engineered pipeline)

ROC-AUC scores for the engineered pipeline confirm that Logistic Regression, Random Forest, and XGBoost all achieve near-identical class separation ability:

Model	ROC-AUC
XGBoost	0.7889
Logistic Regression	0.7875
Random Forest	0.7857
Decision Tree	0.5968

Table 5: ROC-AUC scores on real test set (engineered pipeline)

Key Insights

EDA Findings

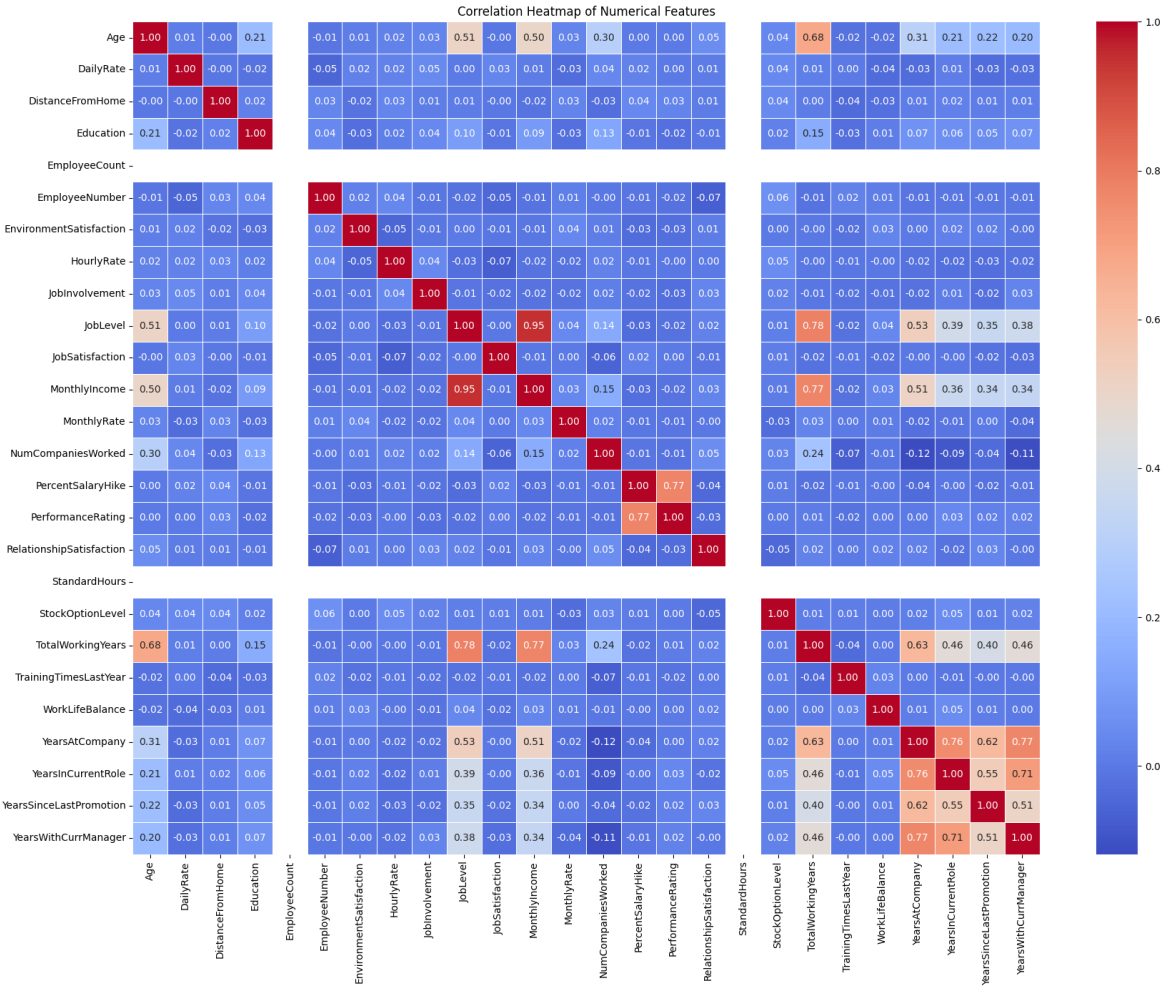

Figure 4: Correlation heatmap of numerical features

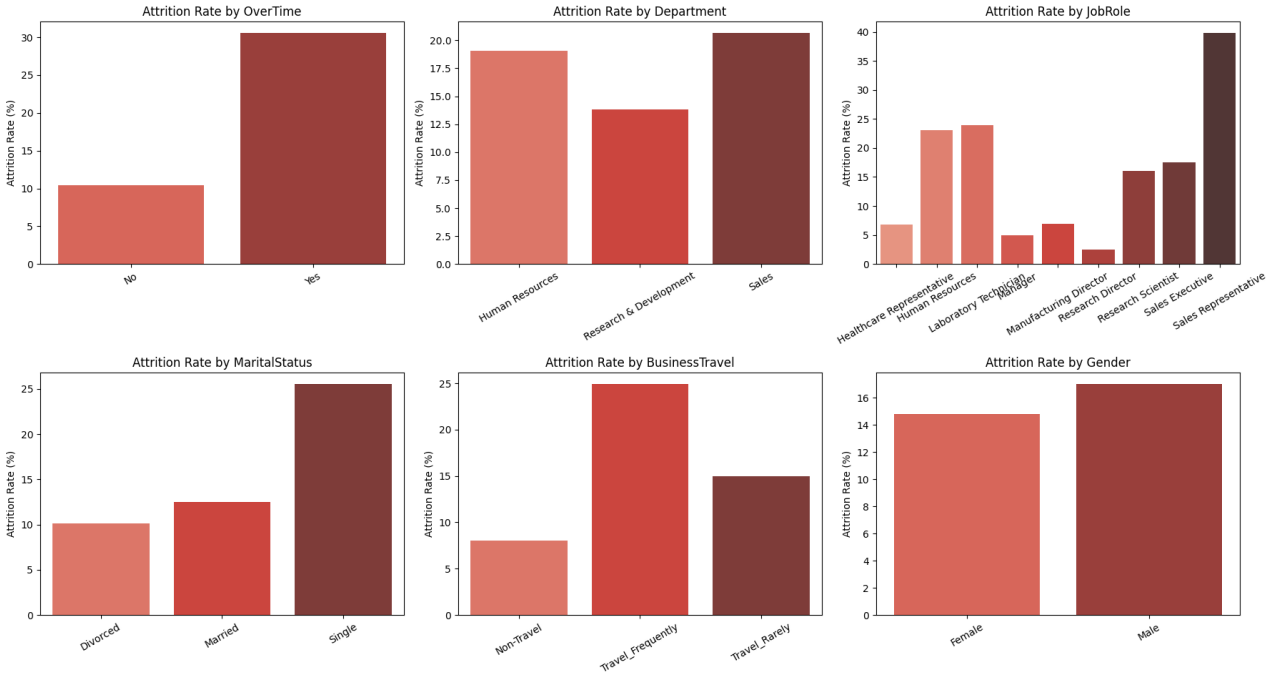

Figure 5: Attrition rate by categorical features

The correlation heatmap (Figure 4) revealed strong multicollinearity between `JobLevel` and `MonthlyIncome` (0.95) and between `TotalWorkingYears` and `MonthlyIncome` (0.77), indicating all three measure employee seniority. The categorical analysis (Figure 5) revealed:

- **OverTime** is the strongest predictor: approximately 30% attrition for overtime employees vs 10% for non-overtime, a 3x difference.
- **Job role** is highly predictive: Sales Representatives had the highest attrition at approximately 40%, while Research Directors and Manufacturing Directors had the lowest at approximately 3 to 5%.
- **Marital status:** Single employees showed 25% attrition vs 12% for married and 10% for divorced employees.
- **Business travel:** Frequent travellers showed 25% attrition vs 8% for non-travel employees.
- **Gender** was the weakest predictor with only a 2 percentage point difference (Male: 17%, Female: 15%).

Feature Importance and SHAP Findings

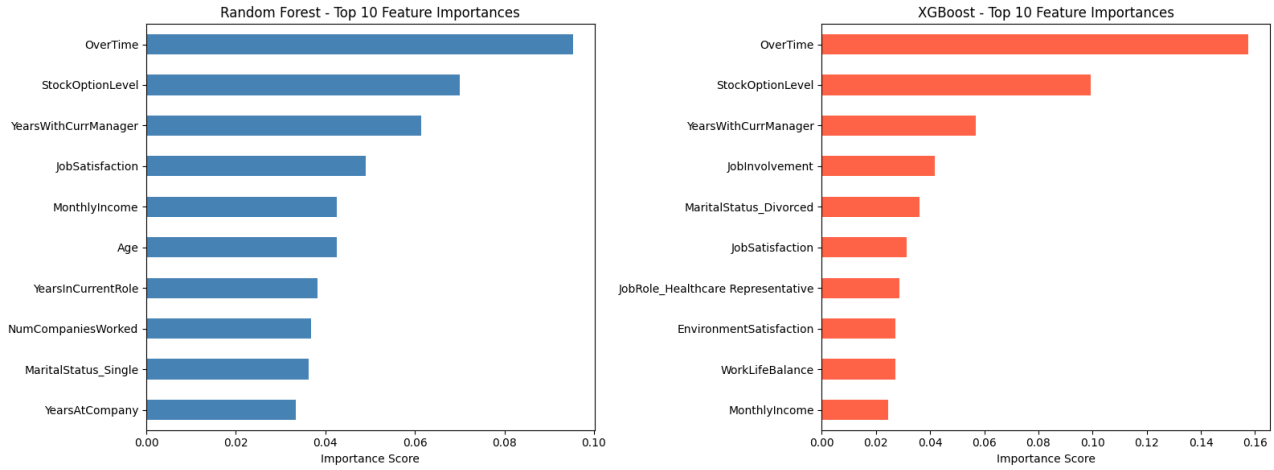

Figure 6: Top 10 feature importances from Random Forest and XGBoost (baseline pipeline)

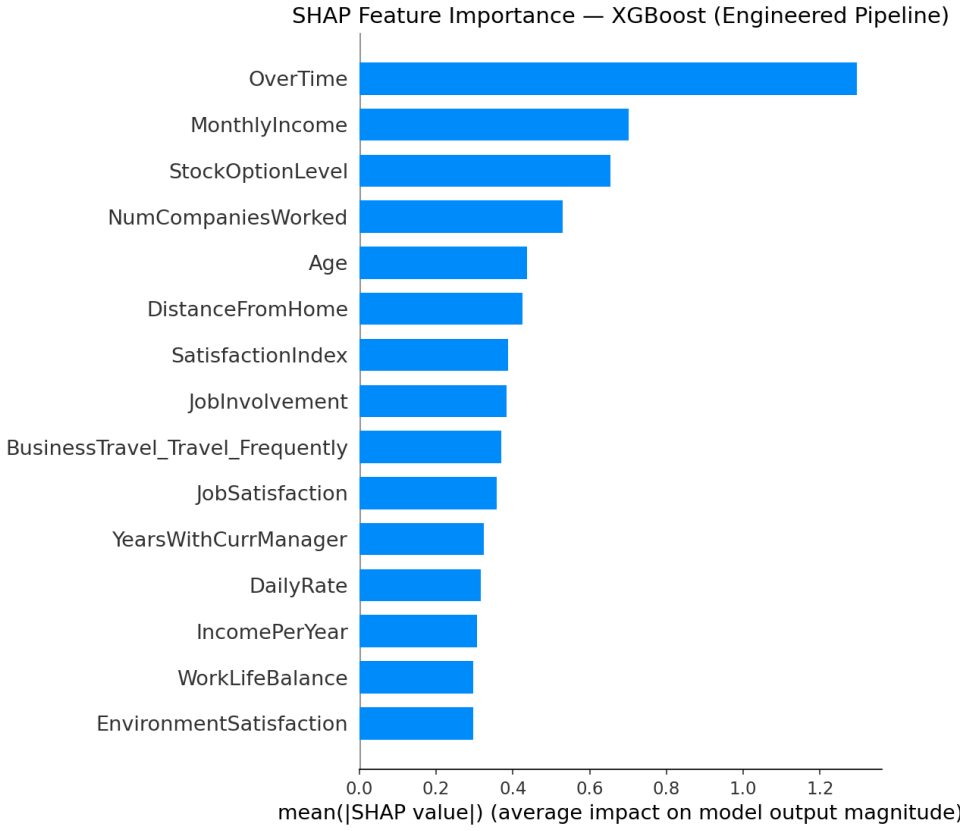

Figure 7: SHAP mean absolute feature impact for XGBoost (engineered pipeline)

Both Random Forest and XGBoost independently ranked `OverTime` as the most important feature, with XGBoost assigning it 15.76% of total feature weight and Random Forest 9.54%. `StockOptionLevel` ranked second in both models, confirming financial retention mechanisms are critical predictors. SHAP analysis (Figure 7) confirmed `OverTime` with a mean absolute SHAP value of 1.30, more than double the second-ranked feature `MonthlyIncome` at 0.70. Two engineered composite features appeared in the SHAP top 15: `SatisfactionIndex` (0.38) and `IncomePerYear` (0.31), directly validating that feature engineering added genuine predictive signal.

Business interpretation: The top three drivers across all experiments are consistent: overwork

(`OverTime`), financial insecurity (`StockOptionLevel`, `MonthlyIncome`, `IncomePerYear`), and workplace instability (`YearsWithCurrManager`). HR intervention should prioritise reviewing overtime policies, revising stock option allocation for junior employees, and monitoring manager transitions.

Model Recommendation

Without threshold tuning: Logistic Regression on the engineered pipeline is recommended for deployment. It achieved the highest default Recall of 0.6170 (catching 29 of 47 actual leavers), maintained stable cross-validated F1 between 0.5221 and 0.5234 across both pipelines, and provides full interpretability through model coefficients without requiring additional tools. XGBoost achieved the highest single-split accuracy (0.8878) but its F1 declined from 0.5217 to 0.4492 under cross-validation, indicating sensitivity to data splitting. Random Forest showed misleadingly high accuracy (0.8469) while catching only 11 of 47 leavers at default threshold.

With threshold tuning: Random Forest at threshold 0.30 achieves the strongest practical performance – Recall of 0.7234 and F1 of 0.5231, catching approximately 34 of 47 actual leavers. This represents a 47 percentage point Recall improvement over its default threshold. For HR teams that can handle a moderate number of false alarms, Random Forest with threshold tuning is the stronger production choice. XGBoost at threshold 0.03 achieves higher Recall (0.7660) but flags nearly all employees as at-risk, which is not operationally practical. The three ROC-AUC scores of 0.79 confirm that Logistic Regression, Random Forest, and XGBoost have near-identical class separation ability – the difference lies in how their probability outputs are calibrated, making threshold tuning the decisive factor.

Challenges Faced

- **Class imbalance and misleading accuracy:** With 84% majority class, a model predicting No for every employee achieves 84% accuracy while catching zero leavers. Random Forest demonstrated this exactly – 84.69% accuracy with only 0.2340 Recall at default threshold, catching just 11 of 47 actual leavers. Evaluation was shifted to Recall and F1-Score throughout.
- **SMOTE placement discipline:** SMOTE was applied only to the training set after splitting, verified explicitly by row count checks (1,176 + 294 = 1,470, zero synthetic contamination in test set). Applying SMOTE before splitting would constitute data leakage, artificially inflating test scores.
- **Single split vs cross-validation conflict:** XGBoost appeared strongest on the single split (F1=0.5217) but weaker under cross-validation (F1=0.4492), demonstrating that a single split can produce results sensitive to random seed. Five-fold stratified cross-validation was essential for reliable model selection.
- **Feature engineering impact varies by model type:** Composite features improved Logistic Regression Recall from 0.5957 to 0.6170 but reduced XGBoost performance. Tree-based models discover nonlinear interactions internally; manually pre-computing them introduces redundancy rather than new information.
- **Hyperparameter tuning score inflation:** GridSearchCV reported F1=0.9389 during tuning on SMOTE synthetic data – an inflated figure excluded from all comparisons. Final evaluation used only the real untouched test set.
- **Threshold selection tradeoff:** Default threshold 0.50 penalised tree-based models heavily on this imbalanced dataset. Threshold optimisation revealed that Random Forest at 0.30 is the strongest practical model, while XGBoost at 0.03 is operationally impractical despite its high Recall. Decision Tree was unresponsive to threshold tuning due to its hard probability outputs.

Technical Stack: Python, Pandas, NumPy, Scikit-learn, XGBoost, Imbalanced-learn (SMOTE), SHAP, Matplotlib, Seaborn — Platform: Kaggle Notebook (CPU)

GitHub Repository: github.com/Ahsan-Neural/employee-attrition-prediction